

Estados estacionarios homogéneos (modelo 3D)

Pipeline SymPy–Newton, `scenarios.json` y post-proceso

Notas técnicas — `Allee/steady_states/`

3 de abril de 2026

Índice

1. Objeto matemático	1
2. Formulaciones de control en el código	2
3. Guía ilustrativa: equilibrios 3D con SymPy y Newton–Raphson	2
3.1. Problema en tres dimensiones	2
3.2. Construcción simbólica de F	2
3.3. Jacobiano analítico	3
3.4. Puente SymPy a NumPy	3
3.5. Algoritmo <code>newton_root_3d</code>	3
3.6. Multirraíz y semillas: dos “rejillas” distintas	4
3.7. Semillas por defecto (<code>default_seeds_3d_control_hill</code>)	4
3.8. Ejemplos de línea de comandos (CLI)	5
3.9. Vista geométrica: cubo de semillas y los dos ejemplos CLI	5
3.10. Deduplicación de soluciones y bandera <code>near_c0_s1_i1</code>	6
3.11. Salida JSON (agrupación por escenario)	6
3.12. Ejemplo numérico reproducible	6
4. Estabilidad lineal local (ODE)	7
5. Filtros físicos (<code>filter_physical_3d</code>)	7
6. Espectro con difusión (inestabilidad de Turing linealizada)	7
7. Flujo de datos en el repositorio	7
7.1. De barrido numérico a <code>scenarios.json</code>	7
7.2. De <code>scenarios.json</code> a CSV de equilibrios	8
7.3. Figuras y espectro a partir del equilibrio 3D	8
8. Resumen	8

1. Objeto matemático

Se considera el subsistema de *reacción* (sin difusión ni dependencia explícita del tiempo) para las concentraciones adimensionales (c, s, i) (tumor, sanos, inmunidad). Un **estado estacionario espacialmente homogéneo** es un punto $\mathbf{x}^* = (c^*, s^*, i^*)$ tal que el campo vectorial de reacción se anula:

$$F_c(c^*, s^*, i^*) = 0, \quad F_s(c^*, s^*, i^*) = 0, \quad F_i(c^*, s^*, i^*) = 0. \quad (1)$$

Numéricamente, (1) es el sistema no lineal $\mathbf{F}(\mathbf{x}) = \mathbf{0}$ con $\mathbf{x} \in \mathbb{R}^3$. Este documento se limita a ese encaje **3D** (tres ecuaciones en (c, s, i)), coherente con `build_equations_3d` y `newton_root_3d`.

Observación 1. *Este equilibrio es un **punto fijo del flujo ODE** $\dot{\mathbf{x}} = \mathbf{F}(\mathbf{x})$. No implica por sí solo interpretación termodinámica; la producción de entropía y los potenciales en el sentido de termodinámica de no equilibrio se construyen en otros módulos del proyecto a partir del PDE y de funcionales sobre los campos.*

2. Formulaciones de control en el código

Es crítico no mezclar ramas:

- **Control tipo Hill** (alineado con el PDE en `cancer_dynamics` y con `build_equations_3d(..., include_hill_control=True)`): ley saturada del tipo Michaelis–Menten generalizada en c e i .
- **Control adaptativo mínimo** (legado): $u = \min(k_u c / (i + \varepsilon_u), u_{\max})$, usado en extracción desde JSON cuando hay control pero el escenario no se clasifica como Hill.
- **Sin control**: término de control ausente en F_i .

La función `scenario_uses_hill_control` en `extract_steady_states_from_scenarios.py` decide la rama coherente con el nombre del escenario y los parámetros `HILL_*`.

3. Guía ilustrativa: equilibrios 3D con SymPy y Newton–Raphson

Esta sección describe el procedimiento tal como está implementado en `steady_states.py` y replicado en `extract_steady_states_from_scenarios.py`, con el nivel de detalle útil para verificar coherencia entre análisis simbólico, álgebra lineal numérica y el PDE.

3.1. Problema en $\mathbb{R}^3$

Se define el vector de estado $\mathbf{x} = (c, s, i)^\top$ y el campo de reacción $\mathbf{F} : \mathbb{R}^3 \rightarrow \mathbb{R}^3$,

$$\mathbf{F}(\mathbf{x}) = \begin{pmatrix} F_c(c, s, i) \\ F_s(c, s, i) \\ F_i(c, s, i) \end{pmatrix}.$$

Un **estado estacionario homogéneo** satisface $\mathbf{F}(\mathbf{x}^*) = \mathbf{0}$. En el flujo ODE asociado $\dot{\mathbf{x}} = \mathbf{F}(\mathbf{x})$, eso es un punto de equilibrio. El objetivo numérico es aproximar $\mathbf{x}^*$ resolviendo el sistema no lineal.

3.2. Construcción simbólica de $\mathbf{F}$ (`build_equations_3d`)

SymPy declara los símbolos (c, s, i) y parámetros $(r_c, r_s, r_d, \alpha, \beta, \delta, \gamma, \eta, \mu, a, \dots)$. El término de crecimiento con Allee **débil** es

$$\text{alle}_{\text{weak}}(c) = r_c c (c - a) (1 - c),$$

y con Allee **fuerte**,

$$\text{alle}_{\text{strong}}(c) = r_c c (1 - c) \frac{c - a}{1 - a}.$$

Las tres componentes (sin fuentes externas aparte del control) son

$$\begin{aligned} F_c &= \text{alle}(c) - c(\alpha s^2 + \beta i^2) - \mu c(\gamma s^2 + \eta i^2), \\ F_s &= r_s s(1 - s) - \gamma c^2 s + \delta i^2 s - \frac{\mu\alpha}{2} c^2 s, \\ F_i &= r_d i(1 - i) + \delta i s^2 - \eta c^2 i - \frac{\mu\beta}{2} c^2 i + u_{\text{ctrl}}. \end{aligned}$$

Si `include_hill_control=True`, se usa $i_+ = \max(i, 0)$ y

$$H_{\text{act}}(c) = \frac{c^{n_c}}{k_c^{n_c} + c^{n_c}}, \quad H_{\text{inh}}(i) = \frac{k_i^{n_i}}{k_i^{n_i} + i_+^{n_i}}, \quad u_{\text{ctrl}} = u_{\text{máx}} H_{\text{act}}(c) H_{\text{inh}}(i).$$

Si no hay control Hill, $u_{\text{ctrl}} = 0$. La expresión $\text{Max}(i, 0)$ en SymPy regulariza la derivada donde $i < 0$ en el dominio simbólico; en puntos interiores con $i > 0$ coincide con i .

3.3. Jacobiano analítico $J(\mathbf{x}) = \partial \mathbf{F} / \partial (c, s, i)$

Se forma $\mathbf{F}$ como vector columna simbólico y se define

$$J(c, s, i) = \frac{\partial \mathbf{F}}{\partial (c, s, i)} = \begin{pmatrix} \partial F_c / \partial c & \partial F_c / \partial s & \partial F_c / \partial i \\ \partial F_s / \partial c & \partial F_s / \partial s & \partial F_s / \partial i \\ \partial F_i / \partial c & \partial F_i / \partial s & \partial F_i / \partial i \end{pmatrix}.$$

En un equilibrio $\mathbf{x}^*$, la matriz $J(\mathbf{x}^*)$ es el generador del flujo linealizado $\dot{\mathbf{x}} \approx J(\mathbf{x}^*)(\mathbf{x} - \mathbf{x}^*)$; sus autovalores determinan la estabilidad local del punto fijo (misma J que en la sección posterior sobre estabilidad ODE).

3.4. Puente SymPy $\rightarrow$ NumPy (`build_numeric_3d`)

El flujo es:

1. Sustituir parámetros numéricos en F_c, F_s, F_i (`.subs`) obteniendo componentes sin símbolos de parámetros.
2. Construir $\mathbf{F}_{\text{vec}} = \text{Matrix}([F_c, F_s, F_i])$ y $J = \mathbf{F}_{\text{vec}}.\text{jacobian}([c, s, i])$ (objeto SymPy 3×3).
3. `lambdify((c,s,i), F_vec, modules='numpy')` produce una función $\mathbf{f}$ que, evaluada en escalares (c, s, i) , devuelve el vector residual (en la práctica se aplanó a un array 1D de longitud 3).
4. El Jacobiano `no` se convierte a función `lambdify` en el bucle de Newton: en cada iteración se evalúa `J.subs({c:x1^k, s:x2^k, i:x3^k})` y se pasa a `numpy.ndarray float`. Así se usan derivadas exactas (en aritmética de máquina) en lugar de diferencias finitas.

3.5. Algoritmo `newton_root_3d`

Dados $\mathbf{f}$ (NumPy), J (SymPy), semilla $\mathbf{x}^0 = (c^0, s^0, i^0)$, tolerancia ε , máximo de iteraciones $N_{\text{máx}}$ y tope de condicionamiento $\kappa_{\text{máx}}$:

1. Para $k = 0, 1, \dots, N_{\text{máx}} - 1$:
 - a) $\mathbf{F}^k \leftarrow \mathbf{f}(c^k, s^k, i^k)$ (reshape a vector 3).
 - b) $J_{\text{num}}^k \leftarrow$ matriz numérica 3×3 obtenida de J sustituyendo $(c, s, i) \leftarrow (c^k, s^k, i^k)$.
 - c) Si algún elemento no es finito, o $\text{cond}(J_{\text{num}}^k) > \kappa_{\text{máx}}$, abortar (`None`).
 - d) Resolver el sistema lineal $J_{\text{num}}^k \boldsymbol{\delta}^k = \mathbf{F}^k$ (equivalente a $\boldsymbol{\delta}^k = (J_{\text{num}}^k)^{-1} \mathbf{F}^k$ si J es invertible).

- e) Actualizar $\mathbf{x}^{k+1} = \mathbf{x}^k - \boldsymbol{\delta}^k$ componente a componente.
- f) Si $\|\boldsymbol{\delta}^k\| < \varepsilon$, devolver $\mathbf{x}^{k+1}$ como raíz.

2. Si no hubo convergencia, devolver **None**.

La convención $\mathbf{x} \leftarrow \mathbf{x} - J^{-1}\mathbf{F}$ coincide con el paso estándar de Newton para $\mathbf{F}(\mathbf{x}) = \mathbf{0}$ al resolver $J\boldsymbol{\delta} = \mathbf{F}$ y restar $\boldsymbol{\delta}$. Valores por defecto en código: $\varepsilon = 10^{-8}$, $N_{\text{máx}} = 80$, $\kappa_{\text{máx}} = 10^{12}$.

3.6. Multirraíz y semillas: dos “rejillas” distintas

Los sistemas admiten **varias** raíces; el método de Newton solo converge **localmente** a una raíz próxima a la semilla $\mathbf{x}^0 = (c^0, s^0, i^0)$. En el código conviven dos ideas que no deben confundirse:

1. **Rejilla factorial de parámetros** (espacio del *modelo*). Función `control_3d_parameter_mesh()`: recorre el producto de listas finitas para

$$(a, r_c, \beta, \delta, \eta, r_d, r_s, \alpha, \gamma)$$

(orden del `itertools.product` en `scan_grid_3d`). Además de $(r_c, \beta, \delta, \eta, r_d, a)$, se barren **también** r_s, α y γ , que antes permanecían fijos en `params_base_3d`. Valores típicos en código: $r_c \in \{5, 0, 6, 0\}$, $\beta, \delta \in \{5, 0, 7, 0\}$, $\eta \in \{3, 0, 5, 0\}$, $r_d \in \{9, 0, 11, 0\}$, $a = 0, 1$; $r_s \in \{12, 0, 14, 0\}$, $\alpha \in \{9, 0, 11, 0\}$, $\gamma \in \{0, 6, 0, 9\}$ (entornos de los valores base 13, 12, 10, 22, 0, 74). Para cada tupla θ se construye $\mathbf{F}_\theta$ y se buscan ceros en $\mathbf{x} = (c, s, i)$. **Ejemplo numérico:** con dos valores en cada eje salvo a (un solo valor), el número de celdas es $1 \times 2^8 = 256$ (antes $1 \times 2^5 = 32$ sin barrer r_s, α, γ). El coste total escala linealmente con ese factor respecto a versiones anteriores del mismo número de semillas.

2. **Semillas en el cubo de estado** $[0, 1]^3$ (espacio de *incógnitas* de Newton). Son puntos iniciales $\mathbf{x}^0$ para el mismo $\mathbf{F}_\theta$ fijo. No modifican las ecuaciones; solo exploran distintas cuencas de atracción del mismo sistema no lineal.

El coste escala como

$$(\# \text{ celdas paramétricas}) \times (\# \text{ semillas}) \times (\text{coste medio de Newton}),$$

salvo que alguna semilla no converja (**None**).

3.7. Semillas por defecto (`default_seeds_3d_control_hill`)

Sin argumentos extra, la lista **legacy** consta de:

- **Puntos interiores** (exploración heurística): p. ej. $(0, 2, 0, 2, 0, 2)$, $(0, 5, 0, 4, 0, 4)$, $(0, 9, 0, 6, 0, 3)$, $(1, 1, 0, 2, 0, 8)$ (nota: $c = 1, 1$ sale del intervalo $[0, 1]$ adimensional; sirve como semilla numérica, no como cota biológica).
- **Región “esquina”** $c \approx 0, s \approx 1, i \approx 1$: p. ej. $(0, 1, 1)$, $(0, 0, 99, 1)$, $(0, 1, 0, 99)$, y variantes con perturbaciones 10^{-6} – 10^{-10} para evitar singularidades numéricas en factores tipo $s(1-s)$, $i(1-i)$.

Opcionalmente (`grid_n ≥ 2`) se añade el conjunto

$$\mathcal{G}_n = \{0, \frac{1}{n-1}, \dots, 1\}^3$$

mediante `numpy.linspace(0, 1, n)` en cada eje y producto cartesiano (`seeds_unit_cube_grid(n)`), es decir $|\mathcal{G}_n| = n^3$ puntos en $[0, 1]^3$. **Ejemplo** ($n = 3$): $\{0, 0, 5, 1\}^3$ tiene 27 ternas; entre ellas $(0, 0, 0)$, $(0, 5, 0, 5, 0, 5)$, $(1, 1, 1)$, $(0, 1, 0)$, etc. Si `include_legacy=True`, se concatena `legacy+Gn` y se **deduplican semillas** casi idénticas (tolerancia euclídea $\sim 10^{-9}$ en código).

3.8. Ejemplos de línea de comandos (CLI)

Desde el directorio `Allee`, el script `steady_states/steady_states.py` acepta:

```
# Ejemplo A: 5^3=125 semillas en [0,1]^3 + lista legacy (deduplicada).
python steady_states/steady_states.py \
    --seed-grid-n 5 --local-only --no-save

# Ejemplo B (más rápido): 3^3=27 nodos + legacy; control Hill con tope u_max.
python steady_states/steady_states.py \
    --umax 0.5 --seed-grid-n 3

# Ejemplo C: solo cubo 6^3=216 semillas (sin legacy).
python steady_states/steady_states.py \
    --seed-grid-n 6 --seed-grid-only --local-only --no-save
```

3.9. Vista geométrica: cubo de semillas y los dos ejemplos CLI

Trabajamos en el espacio de **estados** (c, s, i) con concentraciones adimensionales. Las semillas del cubo son puntos $\mathbf{x}^0$ en el **paralelepípedo**

$$[0, 1]^3 = \{(c, s, i) : 0 \leq c \leq 1, 0 \leq s \leq 1, 0 \leq i \leq 1\}.$$

Geométricamente es un **cubo unitario**: cada eje es una concentración; el origen $(0, 0, 0)$ y el vértice opuesto $(1, 1, 1)$ son esquinas del mismo sólido. Newton parte de cada punto y converge (o no) hacia algún cero de $\mathbf{F}$ cercano: intuitivamente, cada semilla elige una **cuenca** distinta si el sistema es multirraíz.

Malla cartesiana $\mathcal{G}_N$ (opción `-seed-grid-n N`)

Para entero $N \geq 2$, en cada eje se toman los valores

$$0, \frac{1}{N-1}, \frac{2}{N-1}, \dots, 1$$

(`linspace`). El conjunto de semillas es el **producto cartesiano** de esas tres listas: los N^3 **vértices de una rejilla regular** que *rellena* el cubo con nodos equiespaciados. No es un muestreo aleatorio: es un **latizado** determinista. Ejemplos:

- $N = 5$: $5^3 = 125$ nodos; en cada eje los valores son 0, 0,25, 0,5, 0,75, 1. Aparecen todas las combinaciones; por ejemplo $(0, 0, 0)$, $(1, 1, 1)$, $(0,5, 0,25, 0,75)$, etc.
- $N = 6$: $6^3 = 216$ nodos; el paso en cada eje es 0,2 entre valores consecutivos. La malla es **más fina** que con $N = 5$ (más puntos, más cerca entre sí).

Ejemplo A: `-seed-grid-n 5` sin `-seed-grid-only`

Se usan los 125 **nodos** de $\mathcal{G}_5$ más la lista **legacy**. Geométricamente:

- El cubo queda cubierto por la red $5 \times 5 \times 5$.
- Los legacy añaden puntos **fuera de esa malla regular**: p. ej. $(0,2, 0,2, 0,2)$ cae en el interior pero no coincide necesariamente con un nodo de $\mathcal{G}_5$ (los nodos en cada eje son múltiplos de 0,25); el punto $(1,1, 0,2, 0,8)$ tiene $c = 1,1 > 1$, luego queda **fuera** del cubo $[0, 1]^3$ en la dirección de c (sigue siendo semilla numérica válida).

- Varios legacy se concentran cerca de la **esquina** $(0, 1, 1)$ (tumor muy bajo, s e i altos): refuerzo dirigido en una región biológicamente relevante que la malla sola ya toca en el vértice $(0, 1, 1)$ pero no con las mismas perturbaciones finas.

Tras `_dedupe_seed_points`, algunos legacy y nodos del cubo pueden **coincidir** (p.ej. $(0, 1, 1)$ es vértice de $\mathcal{G}_N$ para todo N), reduciendo el número efectivo de semillas por celda por debajo de $125 + 11$.

Ejemplo B: `-seed-grid-n 6 -seed-grid-only`

Solo los 216 **nodos** de $\mathcal{G}_6$, todos **dentro** de $[0, 1]^3$ y sobre la malla uniforme. Geométricamente:

- Mejor **cobertura uniforme** del volumen que con $N = 5$ (más denso).
- No hay puntos legacy: se **pierde** el refuerzo explícito con $(1, 1, \dots)$ fuera del cubo en c , y el conjunto de perturbaciones muy finas junto a $(0, 1, 1)$ que no son nodos de la malla.

Visualización práctica

Para “ver” las semillas, se puede graficar un `scatter3D` de las ternas (c, s, i) (cubo en matplotlib: ejes $[0, 1]$; en otro color los legacy). El listado exacto se obtiene en Python importando `seeds_unit_cube_grid(N)` y `default_seeds_3d_control_hill(grid_n=..., include_legacy=...)` desde `steady_states.py`.

3.10. Deduplicación de soluciones y bandera `near_c0_s1_i1`

Dentro de `scan_grid_3d`, tras cada convergencia exitosa, dos raíces $\mathbf{x}^*$ con $\|\mathbf{x}_1^* - \mathbf{x}_2^*\| < 10^{-3}$ se consideran la misma: no se duplica la fila en el `DataFrame`. Es independiente de la deduplicación de *semillas*.

Además se marca `near_c0_s1_i1=True` si

$$c^* < 0,12, \quad s^* > 0,88, \quad i^* > 0,88$$

(umbrales `_CORNER_*` en código). Es una **etiqueta heurística**, no una restricción impuesta al Newton.

3.11. Salida JSON (agrupación por escenario)

Los catálogos unificados pueden agrupar equilibrios bajo un **name** de **escenario corto** (tipo `strong_mu0_uNo_bajo_umbral`) y listar varios puntos (c^*, s^*, i^*) en `steady_states` con `n_steady_states`, cuando el mismo θ admite varias raíces encontradas desde distintas semillas. El nombre largo que concatena parámetros y estado se reconstruye como **escenario + equilibrium_slug**.

Cada fila del barrido incluye en el `DataFrame` los parámetros cinéticos barridos: además de $r_c, \beta, \delta, \eta, r_d, a$, columnas `rs`, `alpha`, `gamma`. El `equilibrium_slug` numérico incorpora esos valores (junto con $r_c, \beta, \dots$ y (c^*, s^*, i^*)) para evitar colisiones entre celdas distintas del producto factorial. En la salida del pipeline WEAK, el bloque `weak_meta` puede llevar `parameter_mesh` (listas usadas) y `n_parameter_cells` (producto de cardinalidades).

3.12. Ejemplo numérico reproducible

Configuración. Allee WEAK, sin control Hill (`include_hill_control=False`), $\mu = 1$, resto de parámetros los de `params_base_3d` en el repositorio ($r_c = 5,84$, $r_s = 13,12$, $r_d = 10,92$, $\alpha = 10,22$, $\delta = 5,40$, $\beta = 7,6$, $a = 0,1$, $\gamma = 0,74$, $\eta = 5,08$).

Comando de verificación (desde el directorio `Allee`):

```
python steady_states/verify_equilibrium_point.py --allee WEAK --no-hill \
--newton-seed 0.2 0.2 0.2
```

Primera iteración desde $\mathbf{x}^0 = (0,2, 0,2, 0,2)^\top$:

$$\mathbf{F}(\mathbf{x}^0) \approx \begin{pmatrix} -0,09568 \\ 2,09560 \\ 1,71936 \end{pmatrix}, \quad J(\mathbf{x}^0) \approx \begin{pmatrix} 0,3392 & -0,8768 & -1,0144 \\ -0,4680 & 7,8540 & 0,4320 \\ -0,7104 & 0,4320 & 6,4128 \end{pmatrix}.$$

Resolviendo $J(\mathbf{x}^0)\delta^0 = \mathbf{F}(\mathbf{x}^0)$ se obtiene $\delta^0 \approx (2,07638, 0,36450, 0,47358)^\top$ y por tanto

$$\mathbf{x}^1 = \mathbf{x}^0 - \delta^0 \approx (-1,87638, -0,16450, -0,27358)^\top.$$

Obsérvese que un solo paso puede salir del ortante biológicamente habitual ($c, s, i \geq 0$): ilustra el carácter **local** del método y la necesidad de semillas y filtros posteriores.

Raíz devuelta por el iterador completo con la misma semilla: el algoritmo converge numéricamente a un punto próximo al origen ($c, s, i \approx (0, 0, 0)$ con $\|\mathbf{F}\| \sim 10^{-25}$ (equilibrio de extinción trivial en este modelo cinético). Otras semillas conducen a otras raíces (p.ej. hacia $c \approx 1, s \approx 0, i \approx 0$). Las cifras anteriores se reproducen con `build_numeric_3d({mu:1}, allee_type='WEAK', include_hill_control=False)` y un paso manual de Newton como en el código fuente.

4. Estabilidad lineal local (ODE)

Sea $J(\mathbf{x}^*)$ el Jacobiano de $\mathbf{F}$ en un equilibrio. La linealización

$$\dot{\mathbf{x}} \approx J(\mathbf{x}^*)(\mathbf{x} - \mathbf{x}^*)$$

determina la estabilidad local como punto fijo del flujo ODE: si todas las partes reales de los autovalores de $J(\mathbf{x}^*)$ son negativas, el equilibrio es localmente asintóticamente estable; si alguna parte real es positiva, es inestable. Los DataFrames almacenan partes reales e imaginarias de los autovalores.

5. Filtros físicos (filter_physical_3d)

Tras el barrido 3D, `filter_physical_3d` descarta filas fuera de rangos **ad hoc** de plausibilidad biológica y numérica: positividad estricta de c^* y s^* , rango de i^* , cotas superiores en c^* y s^* . Sobre el espectro de $J(\mathbf{x}^*)$, sea $\Lambda_{\max} = \max_j \operatorname{Re} \lambda_j$ la parte real máxima entre los tres autovalores. El filtro exige

$$\Lambda_{\max} < R_{\max}$$

(con $R_{\max} = 120$ por defecto), es decir, se **excluyen** equilibrios con inestabilidad lineal extrema ($\Lambda_{\max} \geq R_{\max}$). **No** se exige $\Lambda_{\max} > 0$: entran tanto equilibrios **linealmente estables** ($\Lambda_{\max} \leq 0$) como **inestables** ($\Lambda_{\max} > 0$) siempre que $\Lambda_{\max} < R_{\max}$. La columna booleana `unstable` del barrido sigue indicando si existe algún autovalor con parte real positiva.

Son **criterios de post-procesado**, no deducciones del modelo continuo. Nota: equilibrios en fronteras con $c^* = 0$ o $s^* = 0$ (p.ej. $(0, 0, 1)$) quedan fuera de este filtro por la positividad estricta; pueden aparecer solo en salidas *raw*.

6. Espectro con difusión (inestabilidad de Turing linealizada)

Para el modelo reacción-difusión linealizado alrededor de un equilibrio homogéneo, se considera la matriz efectiva

$$J(k) = J_{\text{reac}} - \operatorname{diag}(D_c, D_s, D_i) k^2 \quad (2)$$

y sus autovalores $\lambda(k)$ para modos planos de número de onda k . El análisis en `generate_linear_spectra.py` estudia $\text{Re } \lambda(k)$ frente a k : esqueleto del criterio de inestabilidad de Turing (modo $k > 0$ inestable aunque $k = 0$ sea estable).

7. Flujo de datos en el repositorio

7.1. De barrido numérico a `scenarios.json`

Al ejecutar `steady_states/steady_states.py` en modos como `control-3d`:

1. Se genera un catálogo de equilibrios (crudo y filtrado); opcionalmente `steady_states_catalog.json` con `steady_states_raw` y `steady_states_filtered` como listas de escenarios anidados (`name` corto, `n_steady_states`, `steady_states[]`).
2. Con la bandera `-generate-scenarios`, se fusionan hasta `max_scenarios` escenarios en `Allee/scenarios.json` a partir del DataFrame **ya filtrado** por `filter_physical_3d`, usando `create_scenarios_json(..., overwrite=False)` para no sobrescribir escenarios existentes sin control explícito. Cada escenario generado propaga $r_c, \beta, \delta, \eta, r_d, a$ y también `rs`, `alpha`, `gamma` leídos de la fila (con reserva en `common_params` si faltan en datos antiguos).

7.2. De `scenarios.json` a CSV de equilibrios

`extract_steady_states_from_scenarios.py`:

1. Lee `common_params` y la lista `scenarios`.
2. Para cada escenario, reconstruye **F** según Allee débil/fuerte y la rama de control (Hill / mínimo / ninguna).
3. Ejecuta Newton desde muchas semillas; elimina duplicados cercanos; impone condiciones mínimas ($i > 0$, etc.).
4. Si hay varias raíces, aplica **reglas de selección**: para ciertos nombres de escenario, minimiza distancia a valores tabulados; en caso general, prefiere equilibrios “no triviales” (por ejemplo $s^* > 0,1$ o $c^* > 0,5$).
5. Escribe `steady_states_scenarios.csv` (y copias por escenario) en el directorio de resultados configurado por `utils_paths`, no dentro de `Allee/`.

7.3. Figuras y espectro a partir del equilibrio 3D

`generate_figures_from_scenarios.py` usa el CSV de equilibrios 3D y `scenarios.json`. El bloque relevante para el Jacobiano de reacción J_{reac} en (2) es `generate_linear_spectra.py`, que parte de (c^*, s^*, i^*) del sistema 3D.

8. Resumen

El pipeline en `steady_states/` implementa: (i) definición explícita de **F** coherente con el PDE según el escenario; (ii) búsqueda de ceros por Newton con multirraíz vía semillas; (iii) clasificación local vía espectro de J (columna `unstable`); (iv) barrido factorial ampliado con (r_s, α, γ) además de $(r_c, \beta, \delta, \eta, r_d, a)$; (v) filtrado posterior por concentraciones y cota $\Lambda_{\text{máx}} < R_{\text{máx}}$, admitiendo equilibrios estables e inestables; (vi) enlace con `scenarios.json` y derivados para simulación y figuras.

```
pdflatex -interaction=nonstopmode proceso_estados_estacionarios.tex
```